The Canonical Cryptography Bill of Materials (2CBOM)

A Formal Specification for Post-Quantum Cryptography Compliance

Version 1.0

Amin Jafarpour
School of Computing Science, Simon Fraser University

October 29, 2026

Abstract

Cryptography Bill of Materials (CBOM) generators disagree on names, structure, completeness and assessment of the cryptographic assets they report, and the pre-standard IBM CBOM format coexists with CycloneDX 1.6 and 1.7 documents. This specification defines 2CBOM, a canonical model for cryptographic inventories that is strict enough to support post-quantum cryptography (PQC) compliance determinations. 2CBOM fixes (i) a deterministic canonical identity for algorithms, certificates, key material, protocols and software components, (ii) closed vocabularies, (iii) structural closure constraints that make every asset assessable, (iv) a mandatory quantum-security assessment per asset with NIST IR 8547, CNSA 2.0 and FIPS status, (v) a canonical serialisation with content digest and deterministic serial number, and (vi) merge semantics under which the reconciliation of any set of source CBOMs is a commutative, associative and idempotent operation whose result is independent of the order in which the sources are processed. Mappings from CycloneDX 1.5/1.6/1.7 and from the IBM `1.4-cbom-1.0` format are given, together with a lossless-by-annotation projection back to CycloneDX 1.6. A reference implementation, `twocbom`, realises the model as a rule-based expert system.

Contents

1 Introduction and scope **2**
1.1 Motivation . . . 2
1.2 Scope . . . 2
1.3 Conventions . . . 3

2 Overview of the model **3**

3 Core data model **3**
3.1 Vocabularies . . . 3
3.2 Entities . . . 4

4 Canonical identity **5**
4.1 Identifier space . . . 5
4.2 Canonical strings of algorithms . . . 5
4.3 The canonicalisation function . . . 6
4.4 Cipher suites . . . 7
4.5 Identity of the other asset kinds . . . 7
4.6 Subsumption . . . 7

5 Quantum-security assessment model **8**

6 Constraints and conformance profiles **9**

7 Canonical serialisation and integrity **10**

8 Merge semantics **10**
8.1 Assertion model 10
8.2 Rules and the fixpoint 11
8.3 The 2CBOM rule base 11
8.4 The merge operator 12

9 Mappings from source formats **12**
9.1 Format detection 12
9.2 CycloneDX 1.6 and 1.7 13
9.3 IBM pre-standard CBOM (`1.4-cbom-1.0`) 14

10 Projection to CycloneDX 1.6 **15**

11 Security and privacy considerations **15**

A JSON representation **15**

B Knowledge-base excerpt **16**

C Reference implementation **16**

1 Introduction and scope

1.1 Motivation

Regulatory and organisational PQC programmes (for instance the NIST transition timeline of IR 8547, which deprecates 112-bit quantum-vulnerable public-key cryptography after 2030 and disallows all quantum-vulnerable public-key cryptography after 2035, and the CNSA 2.0 suite) require an *auditable inventory* of cryptographic assets. CBOM generators produce such inventories, but in practice the outputs of CBOMkit, cdxgen, cryptobom-forge, cbomkit-theia and earlier IBM tooling differ in at least four ways:

1. *Naming*: `AES128-GCM`, `AES/GCM/NoPadding`, `aes-128-gcm` and the OID `2.16.840.1.101.3.4.1.6` denote the same algorithm.
2. *Structure and version*: CycloneDX 1.6 `cryptographic-asset` components with `evidence.occurrences` versus IBM `crypto-asset` components with `detectionContext` and camelCase enumerations.
3. *Completeness*: one tool sees the key size, another the mode of operation, a third only the family.
4. *Assessment*: security levels copied from different tables, or absent.

A compliance decision taken over such inputs is neither reproducible nor explainable.

1.2 Scope

2CBOM specifies (a) a data model and closed vocabularies; (b) canonical identity; (c) well-formedness and conformance constraints in two profiles, *core* and *strict*; (d) a quantum-security assessment model; (e) a canonical serialisation; (f) merge semantics; (g) mappings from and to existing formats. It does not specify how assets are detected.

1.3 Conventions

The key words **MUST**, **MUST NOT**, **SHOULD** and **MAY** are to be interpreted as in RFC 2119. Σ^* denotes finite strings over the Unicode alphabet; $\perp$ denotes an absent value; $f : X \rightharpoonup Y$ denotes a partial function; $\mathcal{P}(X)$ the power set of X; $\mathcal{P}^+(X)$ the non-empty subsets. JSON member names are written in `monospace`. Identifiers are compared byte-wise after Unicode normalisation form C.

2 Overview of the model

A 2CBOM document is a finite, typed, acyclic-in-identity structure $D = (hdr, \mathcal{L}, \mathcal{C}, \mathcal{A}, \mathcal{R}, \mathcal{F}, \Gamma)$ consisting of a header, *lineage* $\mathcal{L}$ (the sources and the decisions taken while merging them), software *components* $\mathcal{C}$, cryptographic *assets* $\mathcal{A}$, typed *relations* $\mathcal{R}$ between components and assets, *findings* $\mathcal{F}$ produced by conformance checking, and a *conformance* record Γ. Every entity carries a canonical identifier from the identifier space $\mathcal{K}$ (Section 4); every asset carries evidence, a set of contributing sources and an assessment (Section 5). The document is serialised canonically and sealed with a digest (Section 7). Section 8 defines how a set of source CBOMs is turned into a 2CBOM document.

3 Core data model

3.1 Vocabularies

Definition 3.1 (Closed vocabularies)**.** The following finite sets are *closed*: a document is non-conformant if an enumerated attribute carries a value outside the corresponding set. Each set is a superset of the CycloneDX 1.6 enumeration of the same purpose, so that every value has a CycloneDX image (Section 10).

V_{type}	{algorithm, certificate, related-crypto-material, protocol}
V_{prim}	{drbg, mac, block-cipher, stream-cipher, signature, hash, pke, xof, kdf, key-agree, kem, ae, combiner, other, unknown}
V_{fn}	{generate, keygen, encrypt, decrypt, digest, tag, keyderive, sign, verify, encapsulate, decapsulate, other, unknown}
V_{mode}	{cbc, ecb, ccm, gcm, cfb, ofb, ctr, xts, siv, gcm-siv, ocb, eax, cts, kw, kwp, other, unknown}
V_{pad}	{pkcs5, pkcs7, pkcs1v15, oaep, pss, raw, none, zero, iso10126, iso7816, x923, other, unknown}
V_{curve}	{p192, p224, p256, p384, p521, secp256k1, brainpoolp256r1, brainpoolp384r1, brainpoolp512r1, curve25519, curve448, sm2p256v1, other, unknown}
V_{env}	{software-plain-ram, software-encrypted-ram, software-tee, hardware, other, unknown}
V_{cert}	{none, fips140-k-ln ($k \in \{1,2,3\}$, $n \in \{1,\dots,4\}$), cc-eal$n$, cc-eal$n$+ ($n \in \{1,\dots,7\}$), other, unknown}
V_{proto}	{tls, ssh, ipsec, ike, sstp, wpa, other, unknown}
V_{mat}	{private-key, public-key, secret-key, key, ciphertext, signature, digest, initialization-vector, nonce, seed, salt, shared-secret, tag, additional-data, password, credential, token, other, unknown}
V_{state}	{pre-activation, active, suspended, deactivated, compromised, destroyed}
V_{rel}	{uses, implements, provides, negotiates, signed-by, subject-key-algorithm, instantiates, secured-by, depends-on, component-of}
V_{method}	{static-source, static-binary, dynamic, network, configuration, manual, declared, derived, unknown}

$\mathcal{Q}$	{classically-weak, quantum-vulnerable, unknown, quantum-safe} (quantum classes)
V_{ir}	{acceptable, deprecated-after-2030, disallowed-after-2035, disallowed, unknown}
V_{prio}	{immediate, high, medium, low, none, unknown}
V_{sev}	{error, warning, info}

3.2 Entities

Definition 3.2 (Source record)**.** A source record is a tuple $s = (\text{id}_s, \delta_s, \phi_s, v_s, t_s, \tau_s, \mu_s)$ with a source identifier $\text{id}_s = \texttt{src-}h$ where h is the first 12 hexadecimal digits of the SHA-256 digest δ_s of the source bytes, a format $\phi_s \in$ {cyclonedx-json, ibm-cbom-json, cyclonedx-prestandard-json}, the source's own spec version v_s, a tool record $t_s = (\text{name}, \text{version})$, a trust value $\tau_s \in \{0, \ldots, 100\}$ and a default detection method $\mu_s \in V_{\text{method}}$. $\mathcal{S}$ denotes the set of source records of a document. Because id_s is derived from content, the same source file always receives the same identifier.

Definition 3.3 (Evidence record)**.** An evidence record is a tuple $e = (\text{id}_s, t, \mu, \ell, n, o, \sigma, \gamma, c)$: the contributing source and its tool, a detection method $\mu \in V_{\text{method}}$, an optional normalised location $\ell \in \Sigma^* \cup \{\bot\}$ (file path, binary, endpoint), optional line n and offset $o \in \mathbb{N} \cup \{\bot\}$, optional symbol σ and context γ, and a confidence $c \in [0, 1]$. Evidence with $\mu =$ derived **MUST** name in γ the entity and the rule it was derived from. The *key* of e is $(\text{id}_s, \ell, n, o, \sigma, \mu)$; two evidence records with equal keys are the same record.

Definition 3.4 (Algorithm descriptor)**.** Let $\mathbb{F}$ be the set of algorithm *families* of the knowledge base (`aes`, `rsa`, `ml-kem`, `sha2`, ...) and $\mathbb{F}^+ = \mathbb{F} \cup \{f_1 + \cdots + f_k : f_i \in \mathbb{F}, k \geq 2\}$ the families extended by *hybrid combiners*. An algorithm descriptor is a tuple

$$\delta = (f,\ \pi,\ \kappa,\ m,\ p,\ h,\ \rho,\ \chi)$$

with family $f \in \mathbb{F}^+$, parameter set $\pi \in \Sigma^* \cup \{\bot\}$ (key bits, modulus bits, digest bits, or a PQ parameter-set name such as `sha2-128s`), curve $\kappa \in V_{\text{curve}} \cup \{\bot\}$, mode $m \in V_{\text{mode}} \cup \{\bot\}$, padding $p \in V_{\text{pad}} \cup \{\bot\}$, hash h (the canonical string of a hash descriptor, or $\bot$), derived primitive $\rho \in V_{\text{prim}}$ and constituents χ (a sequence of canonical strings; non-empty exactly for hybrids and cipher-suite-shaped assets). The knowledge base assigns to each family the set of *admissible slots* $\text{slots}(f) \subseteq \{\pi, \kappa, m, p, h\}$; a descriptor **MUST** leave inadmissible slots at $\bot$ (`RSA/ECB/...` therefore has no mode).

Definition 3.5 (Asset)**.** An asset is a tuple $a = (\kappa_a, \tau_a, \nu_a, \Lambda_a, P_a, \alpha_a, E_a, S_a, u_a)$ with canonical identifier $\kappa_a \in \mathcal{K}$, type $\tau_a \in V_{\text{type}}$, display name ν_a, alias set $\Lambda_a \subseteq \Sigma^*$ (every raw name that was mapped to κ_a), a typed property record P_a (Definition 3.6), an assessment α_a (Definition 5.1), a non-empty evidence set $E_a \subseteq \mathcal{E}$, the set $S_a \subseteq \mathcal{S}$ of contributing sources, and an *unresolved* flag $u_a \in \{0, 1\}$ that is 1 exactly when the identity could not be canonicalised.

Definition 3.6 (Property records)**.** P_a is determined by τ_a:

algorithm the descriptor slots of δ, the primitive, the crypto functions $\subseteq V_{\text{fn}}$, the classical security level $\lambda_c \in \mathbb{N} \cup \{\bot\}$, the NIST quantum security level $\lambda_q \in \{0, \ldots, 5\} \cup \{\bot\}$, the OID, the governing standards, execution environment, implementation platform, certification levels, pre-standard names, and the constituents χ (as identifiers).

certificate subject, issuer, serial number, `notBefore`, `notAfter` (ISO 8601, UTC), format, extension, fingerprint, and the identifiers of the signature algorithm and of the subject public-key algorithm (through relations `signed-by` and `subject-key-algorithm`).

related-crypto-material type $\in V_{\text{mat}}$, identifier, state, size in bits, format, dates, the algorithm(s) it instantiates (relation `instantiates`), the mechanism securing it (relation `secured-by`) and fingerprint. The material *value* itself **MUST NOT** be present (Section 11).

protocol type $\in V_{\text{proto}}$, normalised version, the set of cipher suites (each a tuple of name, identifier, the identifiers of its constituent algorithms and the flag *kexKnown*), and references to further assets (relation `uses`).

Definition 3.7 (Component)**.** A software component is $c = (\kappa_c, \tau_c, \nu_c, \text{ver}, \text{grp}, \text{purl}, \text{cpe}, \Lambda_c, S_c, r_c)$ with type τ_c in the CycloneDX component-type vocabulary and root flag r_c (the subject of the inventory).

Definition 3.8 (Relation)**.** A relation is $r = (\kappa_1, \kappa_2, \rho, \text{via}, S_r)$ with $\kappa_1, \kappa_2 \in \mathcal{K}$, kind $\rho \in V_{\text{rel}}$, an optional qualifier *via* (the cipher-suite name for `negotiates`) and contributing sources S_r. The key of r is $(\kappa_1, \kappa_2, \rho, \text{via})$.

Definition 3.9 (Finding and conflict)**.** A finding is $(\text{code}, \text{severity} \in V_{\text{sev}}, \text{subject}, \text{message}, \text{rule})$. A conflict record is $(\text{subject}, \text{attribute}, \text{chosen}, \text{candidates}, \text{rule})$ where *candidates* lists every value asserted for the attribute together with its origin, strength, trust and confidence.

Definition 3.10 (Document)**.** A 2CBOM document is a tuple

$$D = (\mathit{ver}, \mathit{profile}, \mathit{sn}, \mathit{gen}, \mathcal{I}, \mathcal{L}, \mathcal{C}, \mathcal{A}, \mathcal{R}, \mathcal{F}, \Gamma, \Sigma_D)$$

where $\mathit{ver} =$ `1.0`; $\mathit{profile} \in \{\text{core}, \text{strict}\}$; sn is the serial number and $\mathcal{I}$ the integrity block (Section 7); gen records generator and timestamp; $\mathcal{L} = (\mathcal{S}, \text{ruleset}, \text{knowledge base}, \text{conflicts})$ is the lineage; $\Gamma = (\mathit{profile}, \text{conformant}, \#\text{errors}, \#\text{warnings}, \#\text{infos})$ and Σ_D a summary.

4 Canonical identity

4.1 Identifier space

Definition 4.1 (Identifier grammar)**.** $\mathcal{K} \subset \Sigma^*$ is the language generated by

```
id   ::= "2cbom:" kind ":" body
kind ::= "alg" | "cert" | "key" | "proto" | "comp"
body ::= canonical-string   (kind = alg)
     ::=  | ("fp:" hashalg ":" hex) | ("dn:" hex) | ("loc:" hex) |
         ("unresolved:" hex)   (kind = cert)
     ::=  | mattype ":" ("fp:" hashalg ":" hex | "id:" hex | "loc:" hex |
         "src:" hex)   (kind = key)
     ::=  | prototype ["-" version]   (kind = proto)
     ::=  | purl | cpe | [group "/"] name ["@" version]   (kind = comp)
```

where `hex` is a lower-case hexadecimal string and `hashalg` $\in$ {sha256, sha1, sha384, sha512, md5}.

4.2 Canonical strings of algorithms

Definition 4.2 (Canonical string)**.** For a descriptor $\delta = (f, \pi, \kappa, m, p, h, \rho, \chi)$ the canonical string is the `--`joined concatenation of the defined slots in the fixed *slot order*

$$[\![\delta]\!] = f \cdot \pi \cdot \kappa \cdot m \cdot p \cdot h,$$

omitting $\perp$ slots, and $\text{id}(\delta) =$ `2cbom:alg:`$[\![\delta]\!]$. Hybrid families are written $f_1{+}f_2$ where each f_i is itself a canonical string (`x25519+ml-kem-768`). Slot values are drawn from the grammar

```
canonical-string ::= family ["-" pset] ["-" curve] ["-" mode] ["-" padding] ["-"
                     hash]
family           ::= token | token "+" family
pset             ::= digits | digits "-" digits | pq-set | variant
pq-set           ::= ("sha2" | "shake") "-" digits ("s" | "f")
hash             ::= "sha1" | "md5" | ("sha2" | "sha3" | "shake" | "blake2b" |
                     ...)  "-" digits
token            ::= [a-z0-9]+ ("-" [a-z0-9]+)*
```

Because the slot order is fixed and each slot is drawn from a closed set, two descriptors are equal iff their canonical strings are equal (the map $\delta \mapsto [\![\delta]\!]$ is injective on descriptors that respect slots(f)).

Example 4.3.
- `2cbom:alg:aes-128-gcm` (from `AES128-GCM`, `AES-128-GCM`, `aes-128-gcm`, OID `2.16.840.1.101.3.4.1.6`);
- `2cbom:alg:rsa-2048-oaep-sha2-256`; `2cbom:alg:ecdsa-p256-sha2-256`; `2cbom:alg:tdea-cbc`;
- `2cbom:alg:rsa-pkcs1v15-sha2-256` (the Java name `SHA256withRSA`, the X.509 name `sha256WithRSAEncryption` and the OID `1.2.840.113549.1.1.11`);
- `2cbom:alg:ml-kem-768` (also `Kyber768`); `2cbom:alg:slh-dsa-shake-256f`; `2cbom:alg:hmac-sha2-256`;
- `2cbom:alg:x25519+ml-kem-768` (hybrid combiner).

4.3 The canonicalisation function

Definition 4.4 (Canonicalisation)**.** Let H be the set of *hint records* (partial maps from {primitive, parameterSetIdentifier, curve, mode, padding, oid} to strings, as carried by structured CBOM fields). The canonicalisation function

$$\text{canon} : \Sigma^* \times H \to (\Delta \times [0,1] \times \mathcal{P}(\Sigma^*)) \cup \{\bot\}$$

maps a raw name and hints to a descriptor, a confidence and a set of notes, or to $\bot$ (*unresolved*). It is defined by the following stages, each of which is deterministic:

1. **OID stage.** If the name (or the hint `oid`) is an OID present in the knowledge base, the descriptor registered for the OID is used; a parsable name may additionally refine slots the OID leaves open.
2. **Composite stage.** Names of the form `TLS_...` or `SSL_...` are cipher suites and yield $f =$ `suite` with constituents from Definition 4.6.
3. **Alias stage.** The name, lower-cased and stripped of separators (hyphen, underscore, slash, space, comma, dot, plus, brackets, colon, semicolon, quotes), is looked up in the whole-name lexicon (`x25519mlkem768`, `rsassapss`, `desede`, `securerandom`, ...).
4. **Segmentation stage.** Otherwise the squashed name is segmented by greedy longest-match over the lexicon of *atoms* (family names and their aliases, hash names with sizes, modes, paddings, curves, the SLH-DSA suffixes `s`/`f`, numbers and stop words such as `with`, `and`, `padding`, `encryption`). Atoms of at most two characters match only on word boundaries, so that no family is read out of an arbitrary word. Characters not covered by any atom form the *residue*.
5. **Interpretation stage.** The primary family is chosen by a fixed priority (KDF > DRBG > MAC > PQC > classical public-key > AEAD > symmetric > hash); numbers are attached as parameter sets according to the family's parameter-set kind; a hash atom becomes the hash slot; `chacha20`+`poly1305` becomes the AEAD family; two key-establishment families form a hybrid; pre-standard names (`kyber`, `dilithium`, `falcon`, `sphincs+`) are mapped to the FIPS families with their parameter-set renaming (Dilithium 2/3/5 $\mapsto$ ML-DSA 44/65/87) and recorded in the notes.
6. **Hint stage.** Hints fill slots the name leaves at $\bot$; when a hint contradicts a name-derived slot the name wins and a `hint-conflict` note is recorded (finding C20).
7. **Convention stage.** Inadmissible slots are dropped; a padding on an AEAD or stream-like mode (gcm, ccm, ctr, ofb, cfb, xts, kw, ...) is dropped; an RSA signature scheme with a hash and no explicit padding is RSASSA-PKCS1-v1_5 (`rsa-pkcs1v15-`h); the primitive is derived from the family and the slots (a block cipher in an AEAD mode is `ae`; RSA with OAEP is `pke`, with PSS or a hash is `signature`; a hybrid is `combiner`).

The confidence is 1 when the residue is empty and no foreign family was ignored, 0.8 when a foreign family was ignored, 0.7 when the residue is non-empty, and 0 for $\bot$.

Theorem 1 (Fixed points of the grammar)**.** *For every descriptor δ that respects* slots(f) *and whose family is in the knowledge base,* $\text{canon}([\![\delta]\!], \emptyset) = (\delta, 1, \emptyset)$.

Proof sketch. $[\![\delta]\!]$ contains only atoms of the lexicon separated by `-`; segmentation recovers the atoms exactly (every slot value is itself an atom or a hash-atom plus number), the family is unique, and the interpretation stage re-assigns each atom to the slot it came from because slot kinds are disjoint (numbers to π, curve atoms to κ, etc.). The reference implementation tests this property over its catalogue of names. □

Remark 4.5. Theorem 1 is what makes identifiers *stable*: re-canonicalising a 2CBOM document (for instance when it is itself fed back as an input) does not change any identifier.

4.4 Cipher suites

Definition 4.6 (Cipher-suite decomposition)**.** An IANA cipher-suite name of the form `TLS_<kex>_<auth>_WITH_<cipher>_<hash>` decomposes into constituents with roles: key exchange (`ECDHE`, `ECDH` ↦ `ecdh`; `DHE`, `DH` ↦ `dh`; `RSA` ↦ `rsa`; `PSK`), authentication (`RSA`, `ECDSA`, `DSS` ↦ `dsa`; absent for `anon`), the cipher c (canonicalised by Definition 4.4, e.g. `3DES_EDE_CBC` ↦ `tdea-cbc`, `NULL` ↦ `null`) and the PRF hash h (`SHA` ↦ `sha1`, `SHA256` ↦ `sha2-256`, ...). TLS 1.3 suites (`TLS_AES_256_GCM_SHA384`, no `WITH`) have no key-exchange constituent; their *kexKnown* flag is false and the key exchange **MUST** be taken from the protocol's references (`cryptoRefArray` groups). Hexadecimal identifiers are resolved through the knowledge base (`0x1301` ↦ `TLS_AES_128_GCM_SHA256`).

4.5 Identity of the other asset kinds

Definition 4.7 (Identity functions and identity bases)**.**

- **Certificate**, in order of preference (H_{24} denotes the first 24 hex digits of SHA-256 and x' the distinguished-name normalisation of x: attribute types upper-cased, spacing collapsed):
  - `2cbom:cert:fp:`$\langle alg \rangle$`:`$\langle hex \rangle$ when a fingerprint is available (basis *fingerprint*);
  - `2cbom:cert:dn:`$H_{24}(\text{JCS}(\text{subject}', \text{issuer}', \text{serial}, \text{notBefore}, \text{notAfter}))$ (basis *distinguished-name*);
  - `2cbom:cert:loc:`$H_{24}(\text{location})$ (basis *location*); otherwise *unresolved*.
- **Key material** (with $\tau \in V_{\text{mat}}$ the material type), in order of preference:
  - `2cbom:key:`τ`:fp:`$\langle alg \rangle$`:`$\langle hex \rangle$ by fingerprint;
  - `2cbom:key:`τ`:id:`$H_{24}(\text{identifier})$ by identifier;
  - `2cbom:key:`τ`:loc:`$H_{24}(\text{JCS}(\text{algorithm}, \text{location}, \text{size}))$ by location;
  - `2cbom:key:`τ`:src:`$H_{24}(\text{source}, \text{ref})$ otherwise (basis *source-local*).
- **Protocol.** id = `2cbom:proto:`τ`-`v with $\tau \in V_{\text{proto}}$ and v the normalised version (`TLSv1.2`, `tls1.2`, `1.2` ↦ `1.2`; SSL versions are written `ssl3.0`).
- **Component.** `2cbom:comp:` followed by the package URL without qualifiers, else the CPE, else `[group/]name[@version]` with normalised tokens.

Identity bases are totally ordered, fingerprint > identifier > distinguished-name > location > source-local > unresolved; the strongest available basis **MUST** be used, and bases below *distinguished-name* raise finding C15.

4.6 Subsumption

Definition 4.8 (Specificity and subsumption)**.** For descriptors of the same family, $\delta \sqsubseteq \delta'$ (δ' *extends* δ) iff every slot defined in δ has the same value in δ' and δ' defines at least one further slot. A raw detection d with descriptor δ is *subsumed* by a raw detection d' with descriptor $\delta' \sqsupset \delta$ iff d and d' carry evidence with the same normalised location and the same line. A certificate detected by distinguished name is subsumed by a certificate detected by fingerprint with the same name key. A subsumed detection contributes its raw name (as alias), its evidence and its tool-reported attributes to the subsuming asset but not its identity, and no asset is created for it. When several

detections subsume d, the most specific (most defined slots), then the lexicographically smallest identifier, is chosen.

5 Quantum-security assessment model

Definition 5.1 (Assessment)**.** An assessment is $\alpha = (q, \lambda_q, \lambda_c, \iota, \gamma, \varphi, \pi, \text{rationale})$ with quantum class $q \in \mathcal{Q}$, NIST quantum security level $\lambda_q \in \{0, \ldots, 5\} \cup \{\bot\}$ (0: broken by Shor's algorithm; 1–5: the NIST PQC security categories, $1 \equiv$ AES-128 key search, $2 \equiv$ SHA-256 collision, $3 \equiv$ AES-192, $4 \equiv$ SHA-384 collision, $5 \equiv$ AES-256), classical security strength $\lambda_c \in \mathbb{N} \cup \{\bot\}$ (NIST SP 800-57 bits), NIST IR 8547 status $\iota \in V_{\text{ir}}$, CNSA 2.0 membership $\gamma \in \{0, 1\}$, FIPS approval $\varphi \in \{0, 1, \bot\}$, migration priority $\pi \in V_{\text{prio}}$ and a list of human-readable rationale strings, each produced by a named rule.

Definition 5.2 (Order on quantum classes)**.** $\mathcal{Q}$ is totally ordered by *badness*:

$$\text{classically-weak} < \text{quantum-vulnerable} < \text{unknown} < \text{quantum-safe},$$

and $\text{worst}(X) = \min X$ for $X \subseteq \mathcal{Q}$, $\text{worst}(\emptyset) = \text{unknown}$. Placing *unknown* below *quantum-safe* is the central strictness choice of 2CBOM: an aggregate with an unassessed constituent is never reported safe.

Definition 5.3 (Level function)**.** The knowledge base defines $\lambda(f, \pi, \kappa, h) = (\lambda_c, \lambda_q)$ per family and parameter set (SP 800-57 Table 2, FIPS 203/204/205, SP 800-208); for families whose strength is inherited from a hash (HMAC, HKDF, PBKDF2, LMS/XMSS) λ is that of the hash; for elliptic-curve families it is that of the curve. The knowledge base is *authoritative*: tool-asserted levels that differ are kept as candidates and raise finding C13.

Definition 5.4 (Algorithm classification)**.** Let $P_{\text{conf}} = \{\text{pke}, \text{kem}, \text{key-agree}\}$ and $F_{\text{conf}} = \{\text{encrypt}, \text{decrypt}, \text{encapsulate}, \text{decapsulate}, \text{keyderive}\}$ be the confidentiality primitives and functions, and $H_{\text{weak}} = \{\text{sha1}, \text{md5}, \text{md4}, \text{md2}\}$. For an algorithm asset with descriptor δ, crypto functions *fn* and family entry F (whose class is quantum-safe for symmetric, hash and PQC families, quantum-vulnerable for RSA, DSA, DH, ECDH, ECDSA, EdDSA, X25519/X448, SM2 and ElGamal, and unknown for families absent from the knowledge base):

$$q = \begin{cases} \text{classically-weak} & \text{if } F.\text{class} = \text{classically-weak or } \lambda_c < 112 \text{ or } h \in H_{\text{weak}} \\ F.\text{class} & \text{otherwise} \end{cases}$$

$$\iota = \begin{cases} \text{acceptable} & q = \text{quantum-safe} \\ \text{disallowed} & q = \text{classically-weak, or } q = \text{quantum-vulnerable} \wedge \lambda_c < 112 \\ \text{deprecated-after-2030} & q = \text{quantum-vulnerable} \wedge \lambda_c = 112 \\ \text{disallowed-after-2035} & q = \text{quantum-vulnerable} \wedge (\lambda_c > 112 \vee \lambda_c = \bot) \\ \text{unknown} & \text{otherwise} \end{cases}$$

$$\pi = \begin{cases} \text{immediate} & q = \text{classically-weak} \\ \text{high} & q = \text{quantum-vulnerable} \wedge (\rho \in P_{\text{conf}} \vee \text{fn} \cap F_{\text{conf}} \neq \emptyset) \\ \text{medium} & q = \text{quantum-vulnerable otherwise} \\ \text{none} & q = \text{quantum-safe} \wedge \gamma = 1 \\ \text{low} & q = \text{quantum-safe} \wedge \gamma = 0 \\ \text{unknown} & q = \text{unknown.} \end{cases}$$

$\gamma = 1$ iff (f, π) is in the CNSA 2.0 set {aes-256, sha2-384, sha2-512, ml-kem-1024, ml-dsa-87, lms, hss, xmss, xmss-mt}; φ is the FIPS approval of the family restricted to approved curves and non-broken hashes. The *high* priority captures exposure to harvest-now-decrypt-later attacks.

Definition 5.5 (Aggregation for composite assets)**.** Let $\mathrm{dep}(a)$ be the set of assets a depends on for its security:

- hybrid combiner: its constituents χ; $q =$ classically-weak if any constituent is, quantum-safe if every PQ constituent is quantum-safe, unknown otherwise; λ_q is the minimum over the PQ constituents, λ_c the minimum over all;
- cipher-suite-shaped algorithm, certificate (signature and subject-key algorithms), key material (the algorithms it instantiates), protocol (all constituents of all suites, all referenced assets including certificates): $q = \mathrm{worst}\{q_b : b \in \mathrm{dep}(a)\}$, $\lambda_q = \min$, $\lambda_c = \min$, $\gamma = \bigwedge$, $\pi =$ worst of the constituent priorities (*medium* for a quantum-vulnerable certificate, whose key authenticates rather than encrypts);
- protocol version policy: SSL 2.0/3.0, TLS 1.0/1.1, SSH 1 and IKEv1 are classically-weak regardless of suites; a TLS 1.3 protocol whose suites have $\mathit{kexKnown} = 0$ and that references no key-establishment asset is *unknown*;
- symmetric key material without an algorithm: by size n, classically-weak if $n < 112$, otherwise quantum-safe with $\lambda_q = 5, 3, 1$ for $n \geq 256, 192, 128$.

Proposition 2 (Monotonicity)**.** worst *is monotone and idempotent; hence adding a constituent to a composite asset can only lower its class, and the assessment of an asset is a function of the assessments of its dependencies. Assessment therefore terminates on the dependency DAG (constituents are never composites of their parents).*

6 Constraints and conformance profiles

Let D be a document, $\mathcal{A}_{\mathrm{alg}}$ the algorithm assets, etc., and let $\mathrm{ev}(a) = E_a$, $\mathrm{rel}_\rho(a) = \{b : (\kappa_a, \kappa_b, \rho, \cdot, \cdot) \in \mathcal{R}\}$.

Constraint C01 (Identity)**.** $\forall a, b \in \mathcal{A} \cup \mathcal{C} : \ \kappa_a = \kappa_b \Rightarrow a = b$*, and in the strict profile* $\forall a \in \mathcal{A}_{\mathrm{alg}} : u_a = 0$. Finding C01

Constraint C02 (Evidence totality and location)**.** $\forall a \in \mathcal{A} : \mathrm{ev}(a) \neq \emptyset$ *(C02), and* $\exists e \in \mathrm{ev}(a)$ *with* $\ell_e \neq \perp$ *(C02b, warning in strict, info in core).*

Constraint C03 (Assessment totality)**.** $\forall a \in \mathcal{A} : \alpha_a \neq \perp$*, and in the strict profile* $q_a \neq \mathit{unknown}$. C03

Constraint C04 (Referential integrity)**.** $\forall(\kappa_1, \kappa_2, \cdot, \cdot, \cdot) \in \mathcal{R} : \kappa_1, \kappa_2 \in \mathrm{id}(\mathcal{A} \cup \mathcal{C})$*, and every* `bom-ref` *used by a source resolves within that source.* C04

Constraint C05 (Certificate closure)**.** $\forall a \in \mathcal{A}_{\mathrm{cert}} : \mathrm{rel}_{\mathit{signed\text{-}by}}(a) \neq \emptyset$ *(C05, error in strict) and* $\mathrm{rel}_{\mathit{subject\text{-}key\text{-}algorithm}}(a) \neq \emptyset$ *(C06, warning).*

Constraint C07 (Protocol closure)**.** $\forall a \in \mathcal{A}_{\mathrm{proto}}$*: a has at least one cipher suite or referenced asset (C07), every suite decomposes or references canonical algorithms (C08), and a TLS 1.3 protocol records its key-exchange groups (C19).*

Constraint C09 (Key-material closure)**.** $\forall a \in \mathcal{A}_{\mathrm{mat}}$ *with type* $\in$ *{private-key, public-key}:* $\mathrm{rel}_{\mathit{instantiates}}(a) \neq \emptyset$ *(error in strict); for secret keys a warning.* C09

Constraint C10 (Parameterisation)**.** $\forall a \in \mathcal{A}_{\mathrm{alg}}$ *with family* f*: every slot in* $\mathrm{requires}(f)$ *is defined unless a slot in* $\mathrm{requiresUnless}(f)$ *is defined (an RSA signature scheme identifier need not carry the modulus size). In the strict profile this is an error for detections with located evidence and a warning for assets implied by cipher-suite names, whose parameters are not part of the name.* C10

Constraint C11 (Closed vocabularies)**.** *Every enumerated attribute takes its value in the vocabulary of Definition 3.1.* C11

Constraint C12 (Temporal validity)**.** *Certificate dates are ISO 8601 and notBefore* $\leq$ *notAfter.* C12

Constraint C13 (Level consistency)**.** *A tool-asserted* λ_c *or* λ_q *that differs from the knowledge base is reported (warning) and overridden.* C13

Constraint C14 (Lineage)**.** $\mathcal{S} \neq \emptyset$*; every source carries digest, format, spec version, tool and trust; every attribute conflict is recorded (C14); pre-standard names are recorded (C16); non-standard spec versions are recorded (C17); secret values are never carried (C18, Section 11).*

Constraint C21 (Determinism)**.** *D is serialised as in Section 7; the digest verifies and the serial number is derived from it.* (validator)

Definition 6.1 (Profiles and conformance)**.** The *core* profile demands C01 (first part), C02, C04, C11, C12, lineage and determinism as errors and reports the remaining constraints as warnings or information. The *strict* profile, intended for PQC compliance, additionally raises C01 (second part), C03, C05, C09 (for asymmetric keys) and C10 (for located detections) to errors. A document is *conformant* to a profile iff it contains no finding of severity *error* under that profile; Γ.conformant records this.

7 Canonical serialisation and integrity

Definition 7.1 (Canonical JSON)**.** The JSON representation of D uses the member names of Appendix A. Members whose value is $\perp$ or an empty collection are omitted. All arrays are sorted: assets and components by identifier, relations by (from, to, kind, via), evidence by its key, findings by (severity rank, code, subject, message), aliases and sources lexicographically. The canonical byte form is JCS(D), the RFC 8785 JSON Canonicalization Scheme (sorted members, no insignificant whitespace, shortest number form, UTF-8); 2CBOM uses no non-integer numbers except evidence confidences, which **MUST** be representable exactly as decimal fractions with at most four digits.

Definition 7.2 (Digest scope, digest and serial number)**.** Let D° be D without the members `integrity`, `serialNumber` and `generated.timestamp`. Then

$$\text{digest}(D) = \texttt{sha256:}\text{hex}(\text{SHA-256}(\text{JCS}(D^\circ))),$$
$$sn(D) = \texttt{urn:uuid:}\text{UUIDv5}(N_{2CBOM}, \text{digest}(D)),$$

with the fixed namespace $N_{2CBOM} = \text{UUIDv5}(\text{NAMESPACE_URL}, \texttt{https://2cbom.org/ns/1.0})$. The integrity block records the algorithm (`sha-256`), the canonicalisation (`RFC8785`), the digest and the scope.

Theorem 3 (Serialisation determinism)**.** *If two documents have equal values of* D° *then they have equal digests and serial numbers; conversely a validator recomputing* digest(D) *detects any modification of* D°*, including modifications of assessments. The timestamp is excluded so that re-generation from the same inputs is byte-comparable.*

8 Merge semantics

8.1 Assertion model

Definition 8.1 (Fact alphabet)**.** A *fact* is an immutable, typed tuple. The fact types are: Source; the raw facts RawComponent, RawAsset(s, r, τ, ν, fields), RawEvidence, RawRelation obtained syntactically from a source; the derived facts Candidate($s, r, \kappa, \ldots$) (a proposed identity), Subsumed, Asset(κ, τ), Component, Member(s, r, κ) (raw-to-canonical membership), Alias,

ATTR$(\kappa, n, v, o, \sigma, \tau_o, c)$ (an attribute *candidate* with value v, origin o, strength σ, trust τ_o and confidence c), RESOLVED$(\kappa, n, v, \text{conflict})$, CONFLICT, EVIDENCE, RELATION, SUITE, ASSESSMENT and FINDING. A *working memory* W is a finite set of facts.

Definition 8.2 (Lifting of sources)**.** For a source s, facts(s) is the set of SOURCE, RAW$*$ facts obtained by the format adapter of Section 9. The adapter is syntactic (renaming, kebab-casing, flattening nested components, normalising locations); it takes no semantic decision. A raw asset without any occurrence receives one RAWEVIDENCE with method *declared*.

8.2 Rules and the fixpoint

Definition 8.3 (Rule)**.** A rule is $R = (\text{name}, \text{salience} \in \mathbb{Z}, \text{when}, \text{then})$ where when(W) enumerates *bindings* (tuples of facts and values for which R applies) and then(W, β) returns a finite set of facts to assert. Rules are *monotone*: they only add facts. A rule **MAY** test the absence of a fact only if every rule that can produce such a fact has strictly higher salience (*stratification*); the single such rule in 2CBOM is materialisation, which tests SUBSUMED facts produced by the subsumption rules at salience 850.

Definition 8.4 (Recognise–act cycle)**.** Given a rule base $\mathcal{R}$ ordered by $(-\text{salience}, \text{registration order})$ and a set Φ of already fired (rule, binding) pairs, one cycle selects the first rule R in that order having a binding $\beta \in \text{when}(W)$ with $(R, \beta) \notin \Phi$, and fires *all* such bindings of R: $W \leftarrow W \cup \bigcup_\beta \text{then}(W, \beta)$, $\Phi \leftarrow \Phi \cup \{(R, \beta)\}$ (*refraction*). The run terminates when no such rule exists. Every firing is appended to a trace $(\text{cycle}, R, \beta, \text{derived facts})$, which is the explanation facility.

Lemma 4 (Termination and uniqueness)**.** *Let* $T_{\mathcal{R}}(W) = W \cup \bigcup_{R \in \mathcal{R}} \bigcup_{\beta \in when(W)} then(W, \beta)$. *If every* then *is deterministic and depends on* W *only through facts, if the number of distinct facts derivable from a finite* W_0 *is finite, and if stratification holds, then the recognise–act cycle started at* W_0 *terminates and its final state is* $\text{lfp}_{W_0}(T_{\mathcal{R}})$*, independently of the registration order of rules of equal salience.*

Proof sketch. Monotonicity and finiteness give termination (each cycle fires at least one new (rule, binding) pair from a finite set). Any firing order reaches the same closure because *then* is a function of W and bindings are never invalidated (no retraction); stratification ensures that the single negation is evaluated on a complete stratum, which makes $T_{\mathcal{R}}$ monotone on the fact lattice; the least fixpoint above W_0 is unique (Knaster–Tarski). □

8.3 The 2CBOM rule base

The rule base is organised in salience bands; Table 2 lists it. Three rule groups carry the semantic content of the merge:

Resolution of attributes (salience 500).

Definition 8.5 (Resolution order)**.** For ATTR candidates of one (κ, n) define the total order

$$a \prec b \iff (\text{rank}(o_a), \sigma_a, \tau_a, c_a, v_a^{-1}, o_a^{-1}) >_{\text{lex}} (\text{rank}(o_b), \sigma_b, \tau_b, c_b, v_b^{-1}, o_b^{-1})$$

with rank(kb) $= 3 >$ rank(derived) $= 2 >$ rank(tool) $= 1$ and v^{-1}, o^{-1} denoting lexicographic comparison of the JCS encoding of the value and of the origin identifier in *ascending* order. RESOLVED$(\kappa, n, \cdot)$ takes the $\prec$-minimum. Multi-valued attributes (crypto functions, certification levels, standards, pre-standard names, cipher suites, references) are resolved by set union. A CONFLICT is recorded when two tool origins assert different values, or when a knowledge-base or derived value overrides a tool value.

Strength σ is the tool's declared specificity for the attribute (from the tool profile, e.g. CBOMkit for mode, padding and parameter set; cbomkit-theia for certificate and key attributes); trust τ is the tool profile's trust; the knowledge base is authoritative for λ_c, λ_q, OID, standards, FIPS status and the family's quantum class; identity slots are *derived* from the canonical descriptor and therefore outrank tools.

Closure (salience 700). Strings naming algorithms inside other assets (`signatureAlgorithm`, `subjectPublicKeyAlgorithm`, `algorithm` of key material, cipher-suite names, hybrid constituents) are canonicalised and materialised as assets with *derived* evidence that names the parent and the rule; references by `bom-ref` are followed within the source. Key size refines a generic family (`rsa` with size 2048 also instantiates `rsa-2048`, finding I02).

Subsumption (salience 850). Definition 4.8, applied before materialisation (stratified).

8.4 The merge operator

Definition 8.6 (Merge)**.** For a finite set of sources $S = \{s_1, \ldots, s_k\}$ and a profile p,

$$\oplus_p(S) = \text{assemble}\Big(\text{lfp}_{W_0}(T_{\mathcal{R}})\Big), \qquad W_0 = \{\textsc{Config}(p)\} \cup \bigcup_i \text{facts}(s_i),$$

where *assemble* builds D from the fixpoint (one asset per Asset fact with its Resolved attributes, evidence, aliases, sources and assessment; relations grouped by key; findings and conflicts collected) and seals it (Section 7).

Theorem 5 (Order independence)**.** *$\oplus_p$ is a function of the* set *S: for any permutation of the sources and any duplication of a source, $\oplus_p$ yields the same D° and hence the same digest and serial number. Equivalently, the binary operation $\oplus_p(S_1 \cup S_2)$ is commutative, associative and idempotent.*

Proof. W_0 is a set union of per-source fact sets, each of which depends only on the source bytes (source identifiers are content-derived, so a duplicated source contributes identical facts). By Lemma 4 the fixpoint is unique and order-free. The resolution order of Definition 8.5 is total and refers only to fact contents (never to arrival order), so every Resolved value is a function of its candidate set; set-union attributes are order-free by construction. *assemble* sorts every collection by content keys. Hence D° is a function of S. Commutativity, associativity and idempotence follow from the corresponding properties of set union. □

Remark 8.7 (What is and is not decided by order)**.** The only place where ties are broken is Definition 8.5; the tie-break is lexicographic on value and origin identifier, both content-derived. Consequently two tools of equal trust and specificity that disagree on a scalar attribute yield a deterministic choice *and* a conflict record, which is the auditable form of "we could not tell".

9 Mappings from source formats

9.1 Format detection

A JSON document is an IBM pre-standard CBOM iff `bomFormat` = `"CBOM"` or `specVersion` starts with `1.4-cbom`; a CycloneDX 1.4 document whose components include type `crypto-asset` is treated the same way. A document with `bomFormat` = `"CycloneDX"` and `specVersion` $\in \{1.5, 1.6, 1.7\}$ is read by the CycloneDX adapter (1.5 and unknown versions with finding C17). Documents without markers are classified by the presence of `cryptoProperties`.

```
procedure MERGE(sources S, profile p):
    W  <- { Config(p) }
    for s in S:                                  # order irrelevant (Theorem 3)
        W <- W U facts(s)                        # syntactic lifting by the format adapter
    Phi <- {}                                     # refraction set
    loop:
        pick the first rule R (by -salience, registration) with an unfired binding
        if none: break
        for every unfired binding b of R:        # fire all activations of R
            W <- W U then_R(W, b); Phi <- Phi U {(R, b)}; append (R, b, new facts) to TRACE
    D <- assemble(W);  D.integrity <- digest(scope(D));  D.serialNumber <- uuid5(N, D.integrity)

    return D, TRACE
```

Figure 1: The merge algorithm.

9.2 CycloneDX 1.6 and 1.7

CycloneDX 1.6/1.7	2CBOM	Note
`metadata.tools.components[0]` (or legacy `tools[]`)	source tool, version	trust from tool profile
`metadata.component`	root component	
`components[].type=cryptographic-asset`	asset	nested components flattened
`cryptoProperties.assetType`	τ_a	identity map
`algorithmProperties.{primitive, parameterSetIdentifier, curve, mode, padding}`	hints to canon	name wins on conflict (C20)
`algorithmProperties.{cryptoFunctions, executionEnvironment, implementationPlatform, certificationLevel, classicalSecurityLevel, nistQuantumSecurityLevel}`	tool attribute candidates	levels overridden by KB (C13)
`cryptoProperties.oid`	OID hint / candidate	
`certificateProperties.{subjectName, issuerName, notValidBefore, notValidAfter, certificateFormat, certificateExtension}`	certificate record	
`certificateProperties.signatureAlgorithmRef, subjectPublicKeyRef`	relations `signed-by`, `subject-key-algorithm`	followed within source
`relatedCryptoMaterialProperties.{type, id, state, algorithmRef, size, format, securedBy, dates}`	material record, `instantiates`, `secured-by`	`value` dropped (C18)

CycloneDX 1.6/1.7	2CBOM	Note
`protocolProperties.{type, version, cipherSuites[], cryptoRefArray}`	protocol record, SUITEs, `negotiates`, `uses`	suites decomposed
`hashes[]`	fingerprint	identity basis
`evidence.occurrences[].{location, line, offset, symbol, additionalContext}`	evidence record	location normalised
`dependencies[].dependsOn`	`uses` (component→asset), `depends-on` (component→component)	
`dependencies[].provides`	`provides`	
`properties[]`	tool properties (kept verbatim)	

CycloneDX 1.7 keeps the 1.6 cryptographic model; the adapter reads both with one code path and passes unknown members through as tool properties.

9.3 IBM pre-standard CBOM (`1.4-cbom-1.0`)

IBM 1.4-cbom-1.0	2CBOM	Note
`bomFormat="CBOM", specVersion="1.4-cbom-1.0"`	source format `ibm-cbom-json`	C17
`components[].type=crypto-asset`	asset	
`assetType=relatedCryptoMaterial`	`related-crypto-material`	
`algorithmProperties.variant`	name (when the component name is empty)	
`primitive` `blockcipher`, `streamcipher`, `keyAgree`	`block-cipher`, `stream-cipher`, `key-agree`	kebab-casing
`implementationLevel`	`executionEnvironment`	
`softwarePlainRam`, …	`software-plain-ram`, …	
`relatedCryptoMaterialType` `privateKey`, …	`type private-key`, …	
`relatedCryptoMaterialProperties`	`instantiates` the canonicalised algorithm, size as parameter-set hint	I02
`algorithm` (string) + `size`		
`certificateProperties.signatureAlgorithm`, `subjectPublicKeyAlgorithm` (strings)	canonicalised and linked (`signed-by`, `subject-key-algorithm`)	
`detectionContext[].{filePath, lineNumbers[], offsets[], keywords[], additionalContext}`	one evidence record per line	keyword as symbol
`confidenceLevel`	evidence confidence	
`dependencies[].dependencyType` `uses` / `implements`	relation kind	

10 Projection to CycloneDX 1.6

A 2CBOM document projects onto a CycloneDX 1.6 BOM as follows: every component and asset becomes a component with `bom-ref` equal to its canonical identifier; descriptor slots map to `algorithmProperties` (modes and paddings outside the CycloneDX enumeration map to `other` and are repeated under `properties` as `2cbom:algorithm:mode` / `2cbom:algorithm:padding`); the hash slot, family, standards and constituents are carried as `2cbom:algorithm:*` properties; assessments as `2cbom:assessment:*` properties; relations `uses`, `depends-on` and `provides` map to `dependencies`, all other kinds to `2cbom:relation:<kind>` properties of the metadata; evidence maps to `evidence.occurrences` with tool and method prefixed to `additionalContext`; lineage, profile, digest and conformance map to metadata properties. The projection is lossy with respect to typed relations and provenance but *lossless by annotation*: every 2CBOM member is recoverable from the properties. The projection's `serialNumber` is UUIDv5 of the 2CBOM digest.

11 Security and privacy considerations

- A 2CBOM document **MUST NOT** contain key material, passwords, tokens or other secret values; adapters discard `relatedCryptoMaterialProperties.value` and record C18. Fingerprints of public artefacts (certificates, public keys) are permitted.
- The digest protects integrity but not authenticity; signatures over JCS(D°) are out of scope but straightforward (CycloneDX attestations or JWS over the canonical bytes).
- Trust values order tool assertions; they are declared in the lineage and **MUST** be reviewed when a tool is upgraded. The knowledge base version is recorded in the lineage for the same reason.
- Evidence locations may reveal repository structure; documents intended for sharing **MAY** strip prefixes (the implementation supports `--strip-prefix`) but **MUST NOT** remove evidence records, since C02 would then fail.
- *Unknown* assessments are reported as such and block strict conformance by design; consumers **MUST NOT** treat an unknown class as safe.

A JSON representation

Member names (JSON Schema `2cbom-1.0.schema.json` of the reference implementation):

```
{ "2cbomVersion": "1.0", "profile": "strict",
  "serialNumber": "urn:uuid:...", "generated": {"timestamp": "...", "generator": {...}},
  "integrity": {"algorithm": "sha-256", "canonicalization": "RFC8785", "digest": "sha256:...",
    "scope": "..."},
  "lineage": {"sources": [{"id": "src-36d615754603", "digest": "sha256:...", "format": "
    cyclonedx-json",
                           "specVersion": "1.6", "tool": {"name": "cbomkit", "version":
    "2.1.0"},
                           "trust": 80, "detectionMethod": "static-source", "assetCount": 6}],
              "ruleset": {"version": "1.0", "firedRules": {...}}, "knowledgeBase": {"version":
    "..."},
              "conflicts": [{"subject": "2cbom:alg:...", "attribute": "...", "chosen": ..., "
    candidates": [...], "rule": "..."}]},
  "components": [{"id": "2cbom:comp:pkg:maven/org.bouncycastle/bcprov-jdk18on@1.78", "type": "
    library", ...}],
  "assets": [{"id": "2cbom:alg:aes-128-gcm", "type": "algorithm", "name": "aes-128-gcm",
              "aliases": ["AES", "AES-128-GCM", "AES128-GCM"],
              "algorithm": {"family": "aes", "primitive": "ae", "parameterSet": "128", "mode":
    "gcm",
                            "cryptoFunctions": ["decrypt", "encrypt"], "classicalSecurityLevel":
     128,
                            "nistQuantumSecurityLevel": 1, "oid": "2.16.840.1.101.3.4.1.6",
```

```
                          "standards": ["FIPS 197", "NIST SP 800-38D"], "executionEnvironment
  ": "software-plain-ram"},
           "assessment": {"quantumClass": "quantum-safe", "nistQuantumSecurityLevel": 1,
                           "classicalSecurityLevel": 128, "nistIr8547": "acceptable", "cnsa2":
   false,
                           "fipsApproved": true, "migrationPriority": "low", "rationale":
  [...]},
           "evidence": [{"source": "src-36d615754603", "tool": "cbomkit", "method": "static-
  source",
                         "location": "src/main/java/com/example/payments/CryptoUtil.java", "
  line": 42,
                         "offset": 23, "context": "Cipher.getInstance(\"AES/GCM/NoPadding\")
  ", "confidence": 1.0}],
           "sources": ["src-36d615754603", "src-6b22f6d672d9", "src-e5fc2a359bba"], "
  unresolved": false}],
 "relations": [{"from": "2cbom:proto:tls-1.2", "to": "2cbom:alg:tdea-cbc", "kind": "negotiates
  ",
              "via": "TLS_RSA_WITH_3DES_EDE_CBC_SHA", "sources": ["src-6b22f6d672d9"]}],
 "findings": [{"code": "2CBOM-P01-CLASSICALLY-WEAK", "severity": "warning", "subject": "2cbom:
  alg:sha1",
             "message": "...", "rule": "check.P01-P02-readiness"}],
 "conformance": {"profile": "strict", "conformant": true, "errors": 0, "warnings": 16, "infos":
   21},
 "summary": {"assetCount": 27, "componentCount": 7, "relationCount": 59,
           "assetsByQuantumClass": {...}, "assetsByMigrationPriority": {...}} }
```


B Knowledge-base excerpt

The algorithm catalogue records, per family: primitive, admissible and required slots, parameter sets with (λ_c, λ_q), quantum class, governing standards, FIPS approval, CNSA 2.0 parameter sets, OIDs and pre-standard names. Examples:

Family	Parameter set	λ_c	λ_q	Class	Standard
aes	128 / 192 / 256	128 / 192 / 256	1 / 3 / 5	quantum-safe	FIPS 197
sha2	256 / 384 / 512	128 / 192 / 256	2 / 4 / 5	quantum-safe	FIPS 180-4
rsa	2048 / 3072 / 4096	112 / 128 / 128‡	0	quantum-vulnerable	FIPS 186-5
ecdsa, ecdh	p256 / p384 / p521	128 / 192 / 256	0	quantum-vulnerable	FIPS 186-5, SP 800-186
ml-kem	512 / 768 / 1024	128 / 192 / 256	1 / 3 / 5	quantum-safe	FIPS 203
ml-dsa	44 / 65 / 87	128 / 192 / 256	2 / 3 / 5	quantum-safe	FIPS 204
slh-dsa	sha2-128s ... shake-256f	128 / 192 / 256	1 / 3 / 5	quantum-safe	FIPS 205
lms, hss, xmss	(hash)	128	2	quantum-safe	SP 800-208
tdea	168	112	0	classically-weak	SP 800-67 (withdrawn)
sha1	–	80	0	classically-weak	SP 800-131A

‡conservative (SP 800-57 lists 3072 → 128 and 7680 → 192). Catalogue entries are data, versioned in the lineage; a compliance programme **SHOULD** review them against the standards in force.

C Reference implementation

`twocbom` (Python 3.10+, no runtime dependencies) implements this specification: `twocbom.canonical` (Sections 4 and 7), `twocbom.ingest` (Section 9), `twocbom.engine` and `twocbom.rules` (Section 8, Table 2), `twocbom.validate` (Section 6) and `twocbom.export` (Section 10). Its test suite checks Theorem 1 over the catalogue of names and Theorem 5 over permutations and duplications of five source CBOMs (CBOMkit, cdxgen, cryptobom-forge, an IBM pre-standard

scanner and a CycloneDX 1.7 scanner), and the command `twocbom explain` replays the trace of Definition 8.4 for any identifier.

Code	Constraint	strict	core
C01-IDENTITY-UNRESOLVED	unresolved algorithm name	error	warning
C02-EVIDENCE-MISSING / -UNLOCATED	no evidence / no located evidence	error / warning	error / info
C03-ASSESSMENT-UNKNOWN	quantum class undetermined	error	warning
C04-DANGLING-RELATION	reference to unknown entity	warning	warning
C05-CERT-SIGNATURE-UNLINKED	certificate without signature algorithm	error	warning
C06-CERT-KEY-UNLINKED	certificate without subject-key algorithm	warning	warning
C07-PROTOCOL-SUITES-MISSING	protocol without suites or references	warning	warning
C08-SUITE-UNRESOLVED	cipher suite not decomposable	warning	warning
C09-KEY-ALGORITHM-UNLINKED	key material without algorithm	error*	warning
C10-PARAMETER-SET-MISSING	required slot undefined	error†	warning
C11-VOCABULARY-VIOLATION	value outside closed vocabulary	error	error
C12-CERT-VALIDITY-INVALID	malformed or inverted validity	error	error
C13-LEVEL-INCONSISTENT	tool level contradicts catalogue	warning	warning
C14-SOURCE-CONFLICT	tools disagree, resolved by precedence	info	info
C15-WEAK-IDENTITY	identity by location / source-local	warning	warning
C16-PRESTANDARD-NAME	pre-standard name mapped	info	info
C17-SPEC-VERSION-NONSTANDARD	non-standard source spec version	info	info
C18-SECRET-VALUE-DROPPED	key material value discarded	warning	warning
C19-PROTOCOL-KEX-UNKNOWN	TLS 1.3 without recorded groups	warning	warning
C20-HINT-MISMATCH	structured field contradicts name	warning	warning
P01 / P02	classically weak / quantum-vulnerable asset	warning / info	warning / info
I01 / I02	subsumption / key-size refinement applied	info	info

Table 1: Finding codes (prefix `2CBOM-`) and severities. *asymmetric key types; †detections with located evidence.

Salience	Rule	Effect
900	normalize.{algorithm, certificate, material, protocol, component}	CANDIDATE identities (Section 4)
850	resolve.subsume-by-location, resolve.subsume-certificate-by-dn	SUBSUMED
800	resolve.materialize	ASSET/COMPONENT, MEMBER, ALIAS, identity ATTRS
790	resolve.attributes, resolve.component-attributes	provenance-tagged ATTRS
785–775	resolve.evidence, resolve.relations, resolve.dangling-relation	EVIDENCE, typed RELATIONS, C04
700	closure.{certificate-signature, certificate-subject-key, material-algorithm, material-secured-by, protocol-suites, protocol-crypto-refs, constituents}	referenced algorithms, SUITES, relations
690	closure.refine-generic-by-material	sized instantiation
600, 590	enrich.catalogue, enrich.hybrid-levels	knowledge-base ATTRS
500	settle.resolve-attribute	RESOLVED, CONFLICT
400–380	assess.{algorithm, certificate, material, protocol}	ASSESSMENT
300–280	check.C01 … check.C20, check.P01-P02	FINDING

Table 2: The 2CBOM rule base (45 rules) of the reference implementation.